

The DMA Streaming Framework: Kernel-Level Buffer Orchestration for High-Performance AI Data Paths

A General-Purpose Buffer Orchestration Layer for High-Performance AI

Marco Graziano
Graziano Labs Corp.
Palo Alto, CA, USA
marco@grazianolabs.com

ABSTRACT

AI transport libraries move bytes efficiently, but they commonly assume that buffers are already correctly allocated, placed, shared, registered, and safe under completion and teardown pressure. This paper presents *dmaplane*, a Linux kernel module that makes this missing layer explicit as *buffer orchestration*. *dmaplane* exposes a stable kernel UAPI via `/dev/dmaplane` and composes ring-based command channels, DMA buffer lifecycle management, dma-buf export for cross-device sharing, a kernel-space RDMA engine, NUMA-aware allocation and verification, credit-based flow control, low-overhead observability, and GPU memory integration via PCIe BAR pinning. We evaluate orchestration sensitivity with measurements of NUMA cross-node penalties at DRAM scale, completion-safe flow control under sustained RDMA load, and GPU BAR mapping tiers versus `cudaMemcpy`. We also demonstrate end-to-end disaggregated inference by transferring KV-cache chunks between two machines using RDMA WRITE WITH IMMEDIATE and reconstructing tensor views on the receiver. RDMA measurements use Soft-RoCE; we distinguish measured results from provider-independent properties by construction.

1 INTRODUCTION

Modern AI systems are limited by host side data movement as often as by accelerator compute. Large language model workloads increasingly rely on point to point transfers between GPUs across hosts, including disaggregated inference where a prefill GPU produces KV cache state and a decode GPU consumes it. Transport systems can move bytes efficiently, but they commonly assume that buffers are already allocated on the correct NUMA node, shareable across devices without copies, registered for RDMA, and safe under completion pressure and teardown. These preconditions are not transport. They are buffer orchestration.

This paper presents **dmaplane**, a Linux kernel module that makes buffer orchestration an explicit kernel layer. *dmaplane* exposes a stable UAPI at `/dev/dmaplane` using `ioctl` and `mmap` plus optional dma-buf file descriptors, and composes ring based command channels, DMA buffer lifecycle management, dma-buf export for cross device sharing, a kernel space RDMA engine, NUMA aware allocation and verification, credit based flow control for completion safety, low overhead observability, and GPU memory integration via PCIe BAR pinning.

The repo source code is at <https://github.com/marcoeg/dmaplane>.

1.1 Goals and non goals

Goals. *dmaplane* has two goals:

- Provide a coherent kernel subsystem for buffer lifecycle, placement, sharing, and flow control, integrated with dma-buf, NUMA placement, RDMA memory registration, and GPU BAR pinning constraints.
- Expose a stable kernel UAPI, based on `ioctl` and `mmap` plus optional dma-buf file descriptors, that higher level transports can consume.

Non goals. *dmaplane* is not intended to provide:

- Collective communication primitives or an NCCL replacement.
- A serving framework, including batching, routing, prefix caching, or scheduler integration.
- Connection management (`rdma_cm`), multi-tenant isolation, or production fault recovery.

1.2 Motivating workloads beyond disaggregated inference

Disaggregated inference is a primary motivation because it requires point to point KV cache movement with receiver notification and bounded completion pressure. The same buffer orchestration layer is required in several adjacent workloads.

Mixture of Experts inference. MoE dispatch and combine routes token batches to expert networks on different devices. Activations become fine grained point to point messages. Steady state throughput depends on staging buffer placement, repeated registration cost, and completion safety under bursty traffic.

Disaggregated training. Training pipelines increasingly separate data parallel and model parallel stages from optimizer and checkpoint services. Gradients, optimizer shards, and activation checkpoints move between devices and hosts without a single global collective barrier. Buffer lifecycle, cross device sharing, and teardown safety are required when multiple DMA engines touch the same buffers.

RL rollout weight transfer. Reinforcement learning systems commonly separate actors producing rollouts from learners updating weights. Periodic weight pushes from learners to many actors stress point to multipoint distribution. Orchestration determines whether weights can be staged once,

shared, and transmitted efficiently without CPU copies and without completion overflow during fanout bursts.

LLM training with weight streaming. In weight streaming architectures, such as Cerebras weight streaming, host side pipelines stage large weight shards and stream them into the accelerator each step. The limiting factors are sustained bandwidth, locality to the correct NUMA node and PCIe root complex, and backpressure correctness. Buffer orchestration governs placement, sharing, and flow control under sustained streaming.

1.3 Contributions

This paper makes four contributions:

- (1) A concrete design and implementation of a kernel level buffer orchestration layer with explicit invariants for concurrency, lifecycle, and teardown (Sections 3 and 4).
- (1) Measured sensitivity results showing where orchestration choices dominate throughput and safety: NUMA placement effects at DRAM scale (Table 14), completion safe flow control under sustained load (Table 13), and GPU BAR mapping tier behavior versus cudaMemCpy (Table 15).
- (1) An end to end disaggregated inference demonstration that transfers KV cache chunks between two machines using RDMA WRITE WITH IMMEDIATE and reconstructs tensor views on the receiver (Section 5, Table 16).
- (1) A practical decomposition into independently testable subsystems with reproducibility guidance and appendices for build, deployment, and test mapping (Appendices A through C).

1.4 Paper organization

Section 2 summarizes required background and positions dmaplane relative to adjacent systems. Section 3 describes architecture and design invariants. Section 4 presents the core mechanisms in dependency order. Section 5 describes the disaggregated inference demo and protocol. Section 6 evaluates the system and discusses limitations. Section 7 concludes. References follow the main body.

2 BACKGROUND AND RELATED WORK

dmaplane relies on existing Linux kernel primitives rather than reimplementing memory management or RDMA transport. This section states only the background required to follow the design and then positions dmaplane relative to neighboring systems.

2.1 Kernel primitives used by dmaplane

DMA allocation and mapping. Small control structures often use `dma_alloc_coherent` to obtain coherent memory and a DMA address for a specific device. Large data buffers typically use page backed allocation (`alloc_pages` and

`alloc_pages_node`) and device specific mapping through scatter gather tables and the DMA mapping API.

NUMA placement. `alloc_pages_node(node, ...)` requests allocation from a specific NUMA node, but can fall back silently to other nodes. Correct placement therefore requires explicit verification after allocation.

dma-buf. The dma-buf framework enables zero copy sharing of DMA capable memory between kernel drivers via an exporter importer contract. A key constraint is that scatter gather tables must be constructed per importer attachment because DMA addresses depend on the importing device and its IOMMU context.

Kernel verbs RDMA. The InfiniBand verbs API in `ib_core` provides a provider independent interface for RDMA. Kernel consumers allocate PDs, CQs, QPs, and register memory regions. Kernel space ownership is required when the memory being registered is not userspace memory.

GPU VRAM integration requires a bridge across the `struct page` boundary and uses NVIDIA peer to peer pinning APIs, with teardown constraints enforced by an asynchronous unpin callback.

2.2 Positioning and closest neighbors

Table 1 positions dmaplane relative to systems that occupy adjacent layers.

NCCL. NCCL [5] implements GPU optimized collective communication and includes internal buffer pool and topology logic. It targets barrier synchronized collectives. dmaplane targets point to point pipelines and provides the kernel level buffer lifecycle and completion safety substrate beneath collective scheduling.

Mooncake and TransferEngine. Mooncake [9] and TransferEngine [3] provide transport abstractions across memory tiers and network providers in userspace. They assume that pinned memory, registration, and placement preconditions are satisfied. dmaplane externalizes and measures those preconditions by implementing kernel side allocation, sharing, and registration steps.

nvidia-peermem. nvidia-peermem [6] bridges GPU BAR pages into the RDMA stack by registering them as memory regions. It addresses a narrow GPU to NIC sharing path. dmaplane incorporates an equivalent pinning and teardown discipline as one subsystem and extends it with NUMA placement, dma-buf export, flow control, and observability.

3 ARCHITECTURE AND DESIGN INVARIANTS

dmaplane is a loadable Linux kernel module that registers one character device and exposes a stable UAPI via `/dev/dmaplane`. The device is a kernel service, not a PCIe device driver. It provides buffer orchestration primitives that other components can consume.

3.1 Architecture overview

Figure 1 shows the system structure. Userspace submits work through `ioctl` and maps buffers via `mmap`. Work executes

Table 1: Capability positioning across the buffer orchestration and transport stack.

Capability	dmaplane	nvidia-peermem	NCCL	Mooncake	TransferEngine
Buffer allocation (coherent, page-backed)	Yes	No	internal	No	No
NUMA-aware placement	Yes	No	topology detection	mentioned	mentioned
dma-buf export or import	Yes	importer only	No	No	No
MR registration (kernel-space)	Yes	Yes (GPU BAR only)	internal	via libibverbs	via libibverbs
Credit-based flow control	Yes	No	per-channel credits	implicit in batching	per-connection
GPU BAR pinning and mapping	Yes	Yes	via CUDA	via CUDA	via CUDA
Cross-machine RDMA transport	loopback and peer QP	No	multi-transport	multi-transport	multi-transport
Kernel tracepoint instrumentation	Yes	No	limited	No	No
Collective operations (AllReduce)	No	No	Yes	No	No
Serving integration (batching, routing)	No	No	No	Yes	No

inside per channel worker threads draining submission rings and posting completions to completion rings. Buffers are named objects referenced by IDs, allowing subsystems to compose without exposing raw pointers across the UAPI.

For receiver notification with remote placement, dmaplane uses RDMA WRITE WITH IMMEDIATE as illustrated in Section 5. Each operation produces a send completion on the sender and a receive completion on the receiver carrying a 32 bit immediate value. Each operation consumes one pre posted receive WR on the receiver.

The complete ioctl map, benchmark commands, and test mapping are provided in the appendices.

3.2 Locking model

dmaplane is concurrent. Multiple userspace threads can issue ioctls concurrently, and each channel has an independent worker thread. The module uses a strict lock ordering:

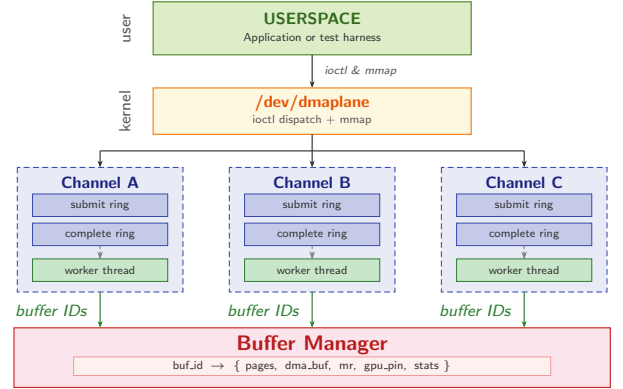
- **dev_mutex** serializes global device operations that can sleep.
- **rdma_sem** is a reader writer semaphore protecting the RDMA context, taking write mode for setup and tear-down and read mode for fast paths.
- **buf_lock** protects the buffer ID map and fast per buffer state transitions.
- **mr_lock** protects the MR registry.

The ordering is **dev_mutex**, then **rdma_sem**, then **buf_lock**, then **mr_lock**.

3.3 Design invariants

The subsystems are coupled by invariants for lifetime, tear-down, and safety.

- **Lock ordering invariant.** Locks are acquired in one global order: **dev_mutex**, then **rdma_sem**, then **buf_lock**, then **mr_lock**. Enforcement: code paths follow the order and do not invert it. Failure prevented: deadlock under concurrent ioctls.

**Figure 1: dmaplane block diagram.**

- **Teardown ordering invariant.** debugfs entries are removed before device teardown, and RDMA tear-down excludes in flight operations by taking **rdma_sem** in write mode. Enforcement: module exit order and semaphore usage. Failure prevented: use after free from debugfs reads or completion processing racing tear-down.
- **mmap lifetime invariant.** A buffer cannot be destroyed while it has active userspace mappings. Enforcement: **mmap_count** accounting on initial **mmap** plus VMA open and close callbacks. Failure prevented: freeing pages still mapped in a process VMA.
- **dma-buf exporter invariant.** Scatter gather tables are constructed per importer attachment. Enforcement: dma-buf exporter **map_dma_buf** builds an SG table for the importing device. Failure prevented: invalid DMA addresses from wrong IOMMU context.
- **GPU unpin callback invariant.** The GPU unpin callback must not block and must not run teardown

under foreign locks. Enforcement: atomic revocation flag and deferred cleanup. Failure prevented: deadlock with NVIDIA driver internal locks.

- **Completion processing invariant.** Completion polling is performed in controlled contexts and quiesced during teardown. Enforcement: polling design and teardown ordering. Failure prevented: asynchronous callbacks executing after resources are freed.

4 CORE MECHANISMS

This section presents the mechanisms that implement the buffer orchestration contract: command channels, buffer lifecycle and sharing, kernel RDMA ownership, completion safe flow control, and GPU integration constraints.

4.1 Channels and ring based dispatch

Each channel provides a submission ring and completion ring plus a worker thread. Userspace submits an entry, the worker executes the operation, and a completion entry returns status and metadata. Rings are fixed size circular buffers with head and tail indices protected by per ring spinlocks. Worker threads sleep on wait queues and wake on submission, and they stop via `kthread_stop` during teardown.

This mechanism provides a stable execution substrate for later subsystems, where the dominant costs come from DMA mapping, RDMA posting, or GPU BAR access rather than ring dispatch.

4.2 Buffer lifecycle, mmap, and dma-buf export

`dmaplane` supports two allocation paths:

- Coherent allocation via `dma_alloc_coherent` for small control structures.
- Page backed allocation via `alloc_pages` or `alloc_pages_node` for large data buffers that require NUMA steering, scatter gather, and MR registration.

Buffers are exposed to userspace without copies by mapping the backing pages into a VMA via `vm_insert_page` in a loop. Buffer teardown must prevent freeing memory while it remains mapped. `dmaplane` tracks `mmap_count` and rejects destroy if the count is non zero. A kernel detail matters: the VMA open callback does not run on the initial `mmap` call, so the initial mapping increments the count explicitly.

For cross device sharing, `dmaplane` exports buffers via `dma-buf`. The exporter provides callbacks to attach and map the buffer for each importing device. A key constraint is per importer scatter gather construction because DMA addresses depend on the importing device and its IOMMU context.

4.3 Kernel RDMA engine ownership

`dmaplane` owns RDMA resources in kernel space because it owns the pages being registered. Userspace verbs APIs register userspace pages, not kernel allocated pages.

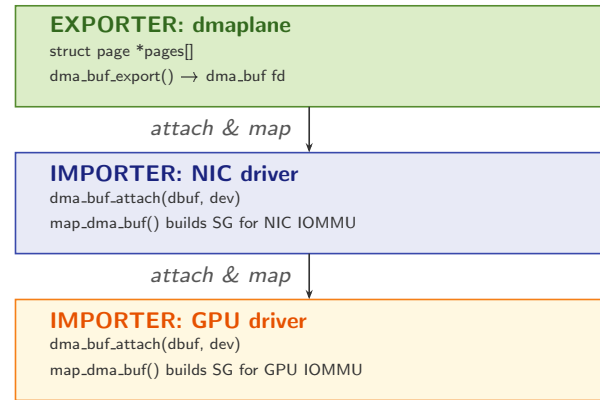


Figure 2: dma-buf export and per importer attach-mapping.

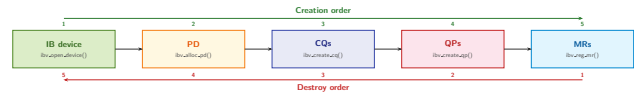


Figure 3: RDMA resource hierarchy.

The verbs resource chain is PD, CQs, QPs, and MRs. `dmaplane` allocates these in order and destroys them in reverse order, flushing QPs by transitioning to error state before destroy. `IB_POLL_DIRECT` is used for CQs so completions are polled explicitly rather than delivered via asynchronous callbacks.

4.4 Completion safe flow control

Completion queue overflow can discard completions and corrupt sender accounting. `dmaplane` enforces a credit invariant that bounds in flight operations by CQ capacity. Credits decrement on post and increment on completion poll, ensuring `in_flight <= max_credits <= CQ_depth` in the configured benchmark paths.

RDMA WRITE WITH IMMEDIATE also consumes receiver side receive WR capacity. A receiver pre posted receive window therefore acts as a second credit type. Safe operation requires bounding in flight WRITE WITH IMMEDIATE operations by both sender completion capacity and receiver notification capacity.

4.5 GPU memory integration constraints

GPU VRAM does not provide `struct page`, so standard kernel DMA and RDMA mapping paths do not apply directly. `dmaplane` uses NVIDIA peer to peer pinning APIs to obtain BAR page descriptors and enforces a non blocking unpinned callback contract by deferring cleanup outside the callback.

Host access to BAR mappings is sensitive to mapping type. Write combining mappings improve host to GPU write

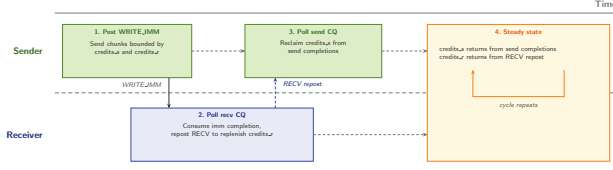


Figure 4: Timeline of send CQ credits and receive window credits in a WRITE WITH IMMEDIATE pipeline.

throughput relative to uncacheable mappings, while GPU to host reads remain limited by non posted PCIe read semantics. The measured tier behavior is reported in Table 15.

5 DISAGGREGATED INFERENCE DEMONSTRATION

This section demonstrates composition of the buffer orchestration layer by transferring KV cache state between two machines and running a decode phase on the receiver.

5.1 Pipeline overview

The demo separates inference into two roles:

- **Prefill machine.** Runs tokenization and the forward pass to produce KV cache. Consolidates KV cache into a pinned GPU staging buffer and transfers it in fixed size chunks to the receiver.
- **Decode machine.** Pre posts receives, accepts RDMA WRITE WITH IMMEDIATE notifications, reconstructs KV cache tensor views over the landing zone, and runs token generation.

The demo uses RDMA WRITE WITH IMMEDIATE so that each chunk is placed into a specific remote address range and also delivers a 32 bit immediate value identifying the chunk.

5.2 Protocol and flow control

Each KV cache transfer is chunked. For each chunk the sender posts `IB_WR_RDMA_WRITE_WITH_IMM` with:

- local SGE referencing the staged chunk
- remote address within the receiver landing zone MR
- receiver rkey
- immediate value encoding of `layer_index` and `chunk_index` (16-bit fields) as implemented in the accompanying artifact

On the receiver, each WRITE WITH IMMEDIATE consumes one pre posted receive WR and produces a receive completion carrying the immediate value. The receive window therefore provides receiver notification credits as defined in Section 4.4, and the sender side flow control provides send CQ credits. The combined bound applies because the verb produces completions on both sides.

Table 2: Disaggregated inference pipeline timing (two EC2 ‘g5.xlarge’, Soft-RoCE).

Phase	Time
Tokenization	1.2 ms
Prefill forward pass	45.3 ms
KV-cache consolidation	0.8 ms
KV-cache transfer	52.1 ms
KV-cache reconstruction	0.003 ms
Time-to-first-token (TTFT)	98.2 ms
Decode throughput	45.3 tok/s
Decode latency (per token)	22 ms average

Table 3: RDMA streaming and flow control results (Soft-RoCE loopback).

Metric	Value
Sustained throughput (10 s, max_credits=64)	1,037 MB/s average
Per-second window range	1,047 to 1,087 MB/s
Window spread	3.8%
CQ overflows	0
Credit stalls	0
Stress configuration (max_credits=4, high=3, low=1)	
Credit stalls (5 s)	72.7 million
CQ overflows	0

A sentinel immediate value signals completion of the transfer, and the receiver verifies that expected chunks arrived before reconstruction.

5.3 Measured timing breakdown

Table 16 reports the measured pipeline timing breakdown for the two instance EC2 demo using Soft-RoCE.

6 EVALUATION AND DISCUSSION

This section presents four result sets that support the buffer orchestration thesis: flow control safety under sustained RDMA load (Table 13), NUMA placement sensitivity at DRAM scale (Table 14), GPU BAR tier behavior versus `cudaMemcpy` (Table 15), and the end to end demo timing breakdown (Table 16). All other microbenchmarks, extended tables, and setup details are provided in the appendices.

6.1 Flow control under sustained streaming

Table 13 reports sustained streaming and stress results under Soft-RoCE loopback. The reported configurations include a high credit configuration and a low credit stress configuration that generates large numbers of stalls while reporting zero CQ overflows.

These results are consistent with the invariant `in_flight <= max_credits <= CQ_depth` for the signaled work request configuration used in the evaluation.

6.2 NUMA placement sensitivity

Table 14 reports cross node CPU memcopy bandwidth on AWS `c5.metal` at 1 MB and 64 MB. The 1 MB case does not show a cross node penalty, while the 64 MB case does, consistent with cache hiding effects at small working sets.

Methodology: based on the accompanying artifact; iteration count and warmup policy are not specified in the draft.

The practical implication for buffer orchestration is that placement errors can be silent and can appear only at DRAM scale buffer sizes.

6.3 GPU memory access tiers

Table 15 consolidates measured tiers for GPU BAR access and `cudaMemcpy` on RTX 5000 Ada, plus measured software RDMA loopback and cross machine behavior from the accompanying artifact.

Methodology: BAR and `cudaMemcpy` tiers are mean of three runs per size as stated in the draft; cross machine tiers are from the two machine configuration described in the draft.

The NIC DMA engine BAR read tiers described in NVIDIA documentation are not measured in this work and must be treated as external claims when referenced.

6.4 End to end demo

Table 16 provides the end to end timing breakdown for the disaggregated inference demo. The largest component in time to first token in the reported run is the KV cache transfer time under Soft-RoCE and Ethernet.

6.5 Limitations and generality

6.5.1 Measured on Soft-RoCE. RDMA measurements in this paper run over Soft-RoCE, a software provider that executes verbs in kernel software and uses CPU memcopy. Results reflect this provider behavior and host CPU scheduling effects.

6.5.2 Provider independent by construction. `dmaplane` uses provider independent kernel verbs APIs and standard completion semantics. The flow control invariant is expressed over completion accounting rules, and the `dma-buf` sharing model follows the kernel framework contract. These properties do not depend on a specific RDMA NIC provider.

6.5.3 Not validated on hardware RDMA. The draft does not include performance measurements on hardware RDMA NICs such as ConnectX or EFA. GPU pinning uses the same peer to peer API used in GPUDirect RDMA environments, but NIC BAR DMA tiers are not measured in this work.

Implementation details, scripts, and additional measurements are provided in the accompanying artifact and appendices.

7 CONCLUSION

This paper presents `dmaplane`, a Linux kernel module that implements buffer orchestration as a first class layer beneath transport and above devices. `dmaplane` provides a stable kernel UAPI and composes mechanisms for buffer lifecycle,

cross device sharing, kernel RDMA ownership, completion safe flow control, NUMA aware placement, observability, and GPU BAR pinning constraints.

Four measured result sets support the framing:

- NUMA placement penalties can be hidden at cache scale and appear at DRAM scale (Table 14).
- Credit based flow control prevents CQ overflow under sustained load in the reported configurations (Table 13).
- BAR mapping tier choice changes host to GPU throughput dramatically while GPU to host reads remain limited (Table 15).
- The end to end demo composes these primitives to transfer KV cache and run decode on the receiver (Table 16).

Future work includes validation on hardware RDMA providers and integration paths where `dmaplane` is consumed by higher level transports or serving frameworks without changing the kernel layer contract.

REFERENCES

- [1] Jonathan Corbet, Alessandro Rubini, and Greg Kroah-Hartman. 2005. *Linux Device Drivers* (3 ed.). O’Reilly Media.
- [2] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. Yu, J. Gonzalez, H. Zhang, and I. Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*.
- [3] Nandor Licker, Kevin Hu, Vladimir Zaytsev, and Lequn Chen. 2025. RDMA Point-to-Point Communication for LLM Systems. arXiv:2510.27656 [cs.DC] <https://arxiv.org/abs/2510.27656>
- [4] NVIDIA Corporation. 2026. GPUDirect RDMA. Documentation. <https://docs.nvidia.com/cuda/gpudirect-rdma/>
- [5] NVIDIA Corporation. 2026. NCCL: NVIDIA Collective Communications Library. Documentation. <https://docs.nvidia.com/deeplearning/nccl/>
- [6] NVIDIA Corporation. 2026. `nvidia-peermem`: NVIDIA Peer Memory Client. Kernel module documentation.
- [7] P. Patel, E. Choukse, C. Zhang, I. Goiri, A. Shah, S. Maleki, and R. Bianchini. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *Proceedings of the 51st International Symposium on Computer Architecture (ISCA)*.
- [8] Ben Pfaff, Justin Pettit, Teemu Koponen, Ethan J. Jackson, Andy Zhou, Jarno Rajahalme, Justin Gross, Alex Wang, Jeremy Stringer, Pravin Shelar, Keith Amidon, and Martin Casado. 2015. The Design and Implementation of Open vSwitch. In *Proceedings of the 12th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*.
- [9] R. Qin, Z. Li, W. He, M. Zhang, Y. Qiu, L. Wang, and J. Lu. 2024. Mooncake: A KVCache-centric Disaggregated Architecture for LLM Serving.
- [10] L. Zheng, L. Yin, Z. Xie, J. Huang, C. Sun, C. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. Gonzalez, C. Barrett, and H. Zhang. 2023. SGLang: Efficient Execution of Structured Language Model Programs.
- [11] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.

A METHODOLOGY

This appendix provides (1) a phase-to-subsystem mapping retained as internal project terminology and (2) a development process note describing AI-assisted development.

Table 4: Cross node memcpy bandwidth on AWS ‘c5.metal’. Values are MB/s.

Buffer size	Node 0 to 0	Node 0 to 1	Node 1 to 0	Node 1 to 1	Cross-node penalty
1 MB	13,284	13,218	12,976	13,051	< 1% (fits in cache)
64 MB	6,778	5,577	5,013	6,095	18% symmetric

Table 5: GPU memory access hierarchy (RTX 5000 Ada). Cross-machine values are over 1 Gigabit Ethernet to a remote NUC.

Access method	Host to GPU (MB/s)	GPU to host (MB/s)	Section
UC BAR (ioremap)	44	6	Accompanying artifact
WC BAR (ioremap_wc, 4 MB)	10,097	107	Accompanying artifact
cudaMemcpy (16 MB)	12,552	13,124	Accompanying artifact
GPU RDMA loopback (rx, 64 KB)	n/a	~20	Accompanying artifact
GPU RDMA cross-machine (1 Gigabit)	n/a	11.6	Accompanying artifact
Derived: 64 KB / 3.2 ms = ~20 MB/s effective for the loopback entry, based on the measured 64 KB loopback latency in the draft.			

A.1 Development process note

The phase structure was used to bound implementation scope, define interfaces, and isolate test surfaces. dmaplane was built using Claude Code as a collaborative tool for implementation, debugging, and test suite generation. The bounded scope of each phase, with a clear specification, a defined interface to previous phases, and an isolated test surface, supported iterative development and verification.

B BUILD AND DEPLOYMENT

This appendix consolidates build and deployment details referenced throughout the paper. The repo source code is at <https://github.com/marcoeg/dmaplane>

B.1 Kernel and configuration requirements

The draft targets a stock Ubuntu kernel (6.5). Required kernel facilities include:

- RDMA core (`ib_core`) and the kernel verbs interface
- Soft-RoCE provider (`rdma_rxe`) for the software provider measurements
- debugfs (`CONFIG_DEBUG_FS`) for debugfs inspection
- ftrace and tracepoints for optional runtime tracing

If debugfs is not mounted or `CONFIG_DEBUG_FS` is disabled, debugfs initialization returns silently and is not treated as fatal.

B.2 Module build and load

The module is built out of tree using Kbuild, the Linux kernel build system. When tracepoints are enabled, the build must satisfy `TRACE_INCLUDE_PATH` and include path requirements so `define_trace.h` can re-include the trace header during macro expansion.

Loading uses standard module tooling:

```
sudo insmod dmaplane.ko
ls -l /dev/dmaplane
```

Unloading requires that userspace closes the device file and releases mappings so teardown invariants are satisfied:

```
sudo rmmod dmaplane
```

B.3 Soft-RoCE bring-up (rxe)

Soft-RoCE is used for the RDMA provider in the draft measurements. A typical bring-up sequence:

```
sudo modprobe rdma_rxe
sudo rdma link add rxe_eth0 type rxe netdev <netdev>
ls /sys/class/infiniband
```

The RDMA device name (for example, `rxe_eth0`) is passed to dmaplane through the RDMA setup ioctl.

B.4 debugfs mount and inspection

Mount debugfs if not already mounted:

```
sudo mount -t debugfs none /sys/kernel/debug
```

Inspect driver state:

```
cat /sys/kernel/debug/dmaplane/stats
cat /sys/kernel/debug/dmaplane/buffers
cat /sys/kernel/debug/dmaplane/rdma
cat /sys/kernel/debug/dmaplane/flow
cat /sys/kernel/debug/dmaplane/histogram
```

B.5 Tracepoint enablement

Enable tracepoints via ftrace:

```
echo 1 > /sys/kernel/debug/tracing/events/dmaplane/dmaplane_rdm
↪ a_post/enable
echo 1 > /sys/kernel/debug/tracing/events/dmaplane/dmaplane_rdm
↪ a_completion/enable
cat /sys/kernel/debug/tracing/trace_pipe
```

B.6 NVIDIA driver constraints and runtime symbol bridge

GPU integration requires the NVIDIA kernel driver and the NVIDIA peer to peer API symbols. Because dmaplane imports GPL only symbols from `ib_core`, it does not take a static link dependency on `nvidia.ko`. Instead, it resolves symbols at runtime using `symbol_get` and releases them using `symbol_put`. If symbols are unavailable, GPU related ioctls return `-ENODEV` and GPU features are disabled.

C TEST AND OBSERVABILITY MAP

This appendix summarizes the test surface and observability mechanisms described in the draft and referenced by the main body. Where specific filenames are not provided in the accompanying artifact, entries are marked as not specified.

C.1 Test suite map

Table C.1. Test suite map by subsystem.

Methodology: derived from the paper text and artifact descriptions; specific script and binary names are not fully specified in the paper.

C.2 Observability summary

dmaplane provides two observability paths:

- **Tracepoints.** Optional kernel tracepoints for events such as RDMA posting, completions, and flow stalls. When disabled, tracepoints compile to near no op behavior.
- **debugfs.** Read-only debugfs files under `/sys/kernel/debug/dmaplane/` support live inspection of counters, buffers, RDMA state, flow state, and the latency histogram. The histogram output format is described in this paper.

The draft removes unconfirmed histogram bucket count tables and retains the mechanism and output format.

D GLOSSARY

ACPI

Advanced Configuration and Power Interface. Firmware interface that provides tables used by the OS, including NUMA topology.

BAR

Base Address Register. PCIe mechanism that maps a device memory region into the host physical address space.

BAR1

NVIDIA term for the PCIe BAR aperture that exposes portions of GPU VRAM to the host.

CAS

Compare and swap. Atomic primitive used to update shared values without locks.

CBQ

Completion queue. See CQ.

CQ

Completion Queue. RDMA data structure that receives Work Completions for completed operations.

CQE

Completion Queue Entry. A single completion record placed into a CQ.

CPU

Central Processing Unit.

CUDA

NVIDIA Compute Unified Device Architecture. Platform and programming model for NVIDIA GPUs.

DMA

Direct Memory Access. Mechanism where a device reads or writes memory without CPU copying.

DRAM

Dynamic Random Access Memory. System main memory.

EFA

Elastic Fabric Adapter. AWS network interface supporting high performance communication, including RDMA like semantics.

ENA

Elastic Network Adapter. AWS virtual network interface used on many EC2 instances.

GID

Global Identifier. RDMA addressing identifier used in RoCE and InfiniBand.

GPU

Graphics Processing Unit.

HCA

Host Channel Adapter. RDMA capable network adapter used in InfiniBand and RoCE environments.

HBM

High Bandwidth Memory. GPU attached memory (for example HBM2e) with very high bandwidth relative to DRAM.

IB

InfiniBand. RDMA interconnect architecture and related kernel subsystem interfaces.

IOMMU

Input Output Memory Management Unit. Hardware that translates device DMA addresses to physical memory and enforces access control.

KV

cache Key value cache. Transformer attention state produced during prefill and consumed during decode.

LID

Local Identifier. InfiniBand subnet local address.

LLC

Last Level Cache. Typically the largest shared CPU cache (often L3).

LLM

Large Language Model.

MR

Memory Region. RDMA object authorizing NIC access to a range of memory, identified by lkey and rkey.

NIC

Network Interface Card.

NUMA

Non Uniform Memory Access. System architecture where memory is divided into nodes with different access costs depending on CPU locality.

P2P

Peer to peer. In this paper, typically refers to GPU peer to peer memory pinning and access for third party devices.

PD

Protection Domain. RDMA resource that scopes access permissions for QPs, CQs, and MRs.

PFC

Priority Flow Control. Ethernet mechanism used to reduce packet loss in RoCE deployments.

QP

Queue Pair. RDMA endpoint containing a Send Queue and Receive Queue.

RC

Reliable Connected. RDMA transport type that provides reliable in order delivery.

RDMA

Remote Direct Memory Access. Mechanism enabling one host to read or write memory on another host without remote CPU involvement in the data path.

RECv

WR Receive Work Request. RDMA receive queue entry posted by the receiver to accept incoming SEND or WRITE WITH IMMEDIATE notifications.

SG

Scatter gather. Representation of a buffer as a list of segments, often page sized, used for DMA mapping.

SGE

Scatter Gather Element. One segment descriptor referenced by an RDMA work request.

SGT

Scatter Gather Table. Collection of SG entries describing a buffer.

SLIT

System Locality Information Table. ACPI table describing relative NUMA node distance.

SQ

Send Queue. RDMA queue inside a QP that holds outgoing work requests.

SOSP

ACM Symposium on Operating Systems Principles.

TLP

Transaction Layer Packet. PCIe transaction unit for reads and writes.

TTFT

Time to first token. Time from request start to the first generated token in an inference pipeline.

UPI

Ultra Path Interconnect. Intel inter socket interconnect.

VMA

Virtual Memory Area. Kernel structure representing a contiguous memory mapping in a process.

VRAM

Video RAM. GPU local memory.

WC

Write combining. CPU memory mapping type that buffers and coalesces writes, used for BAR mappings via `ioremap_wc`.

WR

Work Request. RDMA operation descriptor posted to a QP, such as SEND or RDMA WRITE.

Subsystem	Test entry point	Pass condition
Device and ioctl dispatch	not specified	Module loads, <code>/dev/dmabuf</code> present, ioctls return expected codes.
Rings and workers	userspace stress harness	No kernel warnings, no lockdep issues, no data corruption, clean unload.
DMA buffers and mmap	not specified	Create buffer, mmap, write and verify, munmap, destroy succeeds.
dma-buf export	not specified	Export returns dma-buf fd, attach and detach paths do not leak, release callback fires.
RDMA setup and SEND and RECV	ioctl benchmarks	QP setup succeeds, SEND and RECV completes, CQs polled without errors.
MR registration	ioctl MR tests	MR registration returns lkey and rkey when requested, deregistration releases mappings.
NUMA topology	IOCTL_QUERY_NUMA_TOPO	Returns distance matrix and node inventory without kernel warnings.
NUMA memcpy benchmark	IOCTL_NUMA_BENCH	Produces per-cell bandwidth values without kernel warnings.
Flow control	IOCTL_SUSTAINED_STREAM	Zero CQ overflows under configurations in Table 3.
QD sweep	IOCTL_QD_SWEEP	Produces throughput and latency values (draft Table 4).
Instrumentation	debugfs inspection	debugfs files readable when enabled, histogram format produced during streaming.
GPU pin and BAR mapping	IOCTL_GPU_PIN and IOCTL_GPU_BENCH	Pin succeeds when NVIDIA driver present, unpin callback safe.
GPU RDMA loopback	not specified	Loopback transfer verifies bytes match, no kernel warnings.
WRITE WITH IMMEDIATE helpers	IOCTL_RDMA_WRITE_IMM and related ioctls	Receiver polls <code>imm_data</code> and receives expected identifiers.
Disaggregated inference	prefill and decode scripts	Demo produces coherent output and timings consistent with Table 2.